

```

-- Create Database
CREATE DATABASE company_db;

USE company_db;

-- Create Department Table
CREATE TABLE department(
    dept_id INT PRIMARY KEY AUTO_INCREMENT,
    dept_name VARCHAR(50)
);

-- Create Employee Table
CREATE TABLE employee(
    emp_id INT PRIMARY KEY AUTO_INCREMENT,
    emp_name VARCHAR(50),
    salary DECIMAL(10,2),
    dept_id INT,
    manager_id INT,
    FOREIGN KEY(dept_id) REFERENCES department(dept_id)
);

-- Insert Departments
INSERT INTO department(dept_name)
VALUES
('IT'),
('HR'),
('Finance'),
('Sales'),
('Marketing');

-- Insert Employees
INSERT INTO employee(emp_name,salary,dept_id,manager_id)
VALUES
('Rahul',50000,1,NULL),
('Priya',45000,2,1),
('Aman',60000,1,1),
('Neha',55000,3,3),
('Rohit',48000,4,2),
('Simran',52000,NULL,2),
('Karan',65000,5,1);

-- Display Department Table
SELECT * FROM department;

-- Display Employee Table
SELECT * FROM employee;

-- 1. Inner Join
SELECT e.emp_name,d.dept_name
FROM employee e

```

```

INNER JOIN department d
ON e.dept_id=d.dept_id;

-- 2. Left Join
SELECT e.emp_name,d.dept_name
FROM employee e
LEFT JOIN department d
ON e.dept_id=d.dept_id;

-- 3. Right Join
SELECT e.emp_name,d.dept_name
FROM employee e
RIGHT JOIN department d
ON e.dept_id=d.dept_id;

-- 4. Cross Join
SELECT e.emp_name,d.dept_name
FROM employee e
CROSS JOIN department d;

-- 5. Inner Join with Salary
SELECT e.emp_name,e.salary,d.dept_name
FROM employee e
INNER JOIN department d
ON e.dept_id=d.dept_id;

-- 6. Employees in IT Department
SELECT e.emp_name,e.salary
FROM employee e
INNER JOIN department d
ON e.dept_id=d.dept_id
WHERE d.dept_name='IT';

-- 7. Employees with Salary Greater Than 50000
SELECT e.emp_name,d.dept_name,e.salary
FROM employee e
INNER JOIN department d
ON e.dept_id=d.dept_id
WHERE e.salary>50000;

-- 8. Order Employees by Salary
SELECT e.emp_name,d.dept_name,e.salary
FROM employee e
INNER JOIN department d
ON e.dept_id=d.dept_id
ORDER BY e.salary DESC;

-- 9. Department Wise Employee Count
SELECT d.dept_name,
COUNT(e.emp_id) AS TotalEmployees

```

```
FROM department d
LEFT JOIN employee e
ON d.dept_id=e.dept_id
GROUP BY d.dept_name;
```

```
-- 10. Employees Without Department
SELECT e.emp_name
FROM employee e
LEFT JOIN department d
ON e.dept_id=d.dept_id
WHERE d.dept_id IS NULL;
```

```
-- 11. Departments Without Employees
SELECT d.dept_name
FROM department d
LEFT JOIN employee e
ON d.dept_id=e.dept_id
WHERE e.emp_id IS NULL;
```

```
-- 12. Self Join (Employee and Manager)
SELECT
e.emp_name AS Employee,
m.emp_name AS Manager
FROM employee e
LEFT JOIN employee m
ON e.manager_id=m.emp_id;
```

```
-- 13. Self Join with Manager Salary
SELECT
e.emp_name AS Employee,
m.emp_name AS Manager,
m.salary AS ManagerSalary
FROM employee e
LEFT JOIN employee m
ON e.manager_id=m.emp_id;
```

```
-- 14. Employees Managed by Rahul
SELECT
e.emp_name
FROM employee e
INNER JOIN employee m
ON e.manager_id=m.emp_id
WHERE m.emp_name='Rahul';
```

```
-- 15. Average Salary Department Wise
SELECT
d.dept_name,
AVG(e.salary) AS AverageSalary
FROM department d
INNER JOIN employee e
```

```
ON d.dept_id=e.dept_id  
GROUP BY d.dept_name;
```

```
-- Procedure 1  
DELIMITER $$
```

```
CREATE PROCEDURE getEmployee()  
BEGIN  
    SELECT * FROM employee;  
END $$
```

```
DELIMITER ;
```

```
CALL getEmployee();
```

```
-- Procedure 2  
DELIMITER $$
```

```
CREATE PROCEDURE getEmployeeById(IN id INT)  
BEGIN  
    SELECT *  
    FROM employee  
    WHERE emp_id=id;  
END $$
```

```
DELIMITER ;
```

```
CALL getEmployeeById(1);
```

```
-- Procedure 3  
DELIMITER $$
```

```
CREATE PROCEDURE getDepartment(IN did INT)  
BEGIN  
    SELECT *  
    FROM employee  
    WHERE dept_id=did;  
END $$
```

```
DELIMITER ;
```

```
CALL getDepartment(1);
```

```
-- Procedure 4  
DELIMITER $$
```

```
CREATE PROCEDURE highSalary(IN sal DECIMAL(10,2))
```

```
BEGIN
    SELECT *
    FROM employee
    WHERE salary>sal;
END $$

DELIMITER ;

CALL highSalary(50000);

-- Procedure 5
DELIMITER $$

CREATE PROCEDURE totalEmployee()
BEGIN
    SELECT COUNT(*) AS total
    FROM employee;
END $$

DELIMITER ;

CALL totalEmployee();

-- Procedure 6
DELIMITER $$

CREATE PROCEDURE maxSalary()
BEGIN
    SELECT MAX(salary) AS max_salary
    FROM employee;
END $$

DELIMITER ;

CALL maxSalary();

-- Procedure 7
DELIMITER $$

CREATE PROCEDURE avgSalary()
BEGIN
    SELECT AVG(salary) AS avg_salary
    FROM employee;
END $$

DELIMITER ;
```

```
CALL avgSalary();
```

```
-- Procedure 8
```

```
DELIMITER $$
```

```
CREATE PROCEDURE addEmployee(  
  IN ename VARCHAR(50),  
  IN esalary DECIMAL(10,2),  
  IN did INT,  
  IN mid INT  
)  
BEGIN  
  INSERT INTO employee(emp_name,salary,dept_id,manager_id)  
  VALUES(ename,esalary,did,mid);  
END $$
```

```
DELIMITER ;
```

```
CALL addEmployee('Vikas',58000,2,1);
```

```
SELECT * FROM employee;
```

```
-- Procedure 9
```

```
DELIMITER $$
```

```
CREATE PROCEDURE updateSalary(  
  IN id INT,  
  IN sal DECIMAL(10,2)  
)  
BEGIN  
  UPDATE employee  
  SET salary=sal  
  WHERE emp_id=id;  
END $$
```

```
DELIMITER ;
```

```
CALL updateSalary(2,47000);
```

```
SELECT * FROM employee;
```

```
-- Procedure 10
```

```
DELIMITER $$
```

```
CREATE PROCEDURE deleteEmployee(IN id INT)  
BEGIN  
  DELETE FROM employee
```

```
WHERE emp_id=id;
END $$
```

```
DELIMITER ;
```

```
CALL deleteEmployee(8);
```

```
SELECT * FROM employee;
```

```
-- Procedure 11
```

```
DELIMITER $$
```

```
CREATE PROCEDURE deptCount()
BEGIN
SELECT dept_id,
COUNT(*) AS total
FROM employee
GROUP BY dept_id;
END $$
```

```
DELIMITER ;
```

```
CALL deptCount();
```

```
-- Procedure 12
```

```
DELIMITER $$
```

```
CREATE PROCEDURE employeeDept()
BEGIN
SELECT e.emp_name,
d.dept_name
FROM employee e
INNER JOIN department d
ON e.dept_id=d.dept_id;
END $$
```

```
DELIMITER ;
```

```
CALL employeeDept();
```

```
-- Procedure 13
```

```
DELIMITER $$
```

```
CREATE PROCEDURE salaryDesc()
BEGIN
SELECT *
FROM employee
```

```
ORDER BY salary DESC;
END $$
```

```
DELIMITER ;
```

```
CALL salaryDesc();
```

```
-- Procedure 14
DELIMITER $$
```

```
CREATE PROCEDURE noDepartment()
BEGIN
SELECT *
FROM employee
WHERE dept_id IS NULL;
END $$
```

```
DELIMITER ;
```

```
CALL noDepartment();
```

```
-- Procedure 15
DELIMITER $$
```

```
CREATE PROCEDURE managerList()
BEGIN
SELECT
e.emp_name,
m.emp_name AS manager
FROM employee e
LEFT JOIN employee m
ON e.manager_id=m.emp_id;
END $$
```

```
DELIMITER ;
```

```
CALL managerList();
```

```
-- Create Log Table
CREATE TABLE employee_log(
    log_id INT PRIMARY KEY AUTO_INCREMENT,
    emp_name VARCHAR(50),
    action_name VARCHAR(50),
    action_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
-- Trigger 1
DELIMITER $$
```



```
CREATE TRIGGER after_insert
AFTER INSERT
ON employee
FOR EACH ROW
BEGIN
    INSERT INTO employee_log(emp_name,action_name)
    VALUES(NEW.emp_name,'Employee Inserted');
END $$

DELIMITER ;

INSERT INTO employee(emp_name,salary,dept_id,manager_id)
VALUES('Amit',52000,1,1);

SELECT * FROM employee_log;
```

```
-- Trigger 2
DELIMITER $$

CREATE TRIGGER after_delete
AFTER DELETE
ON employee
FOR EACH ROW
BEGIN
    INSERT INTO employee_log(emp_name,action_name)
    VALUES(OLD.emp_name,'Employee Deleted');
END $$

DELIMITER ;

DELETE FROM employee
WHERE emp_id=3;

SELECT * FROM employee_log;
```

```
-- Trigger 3
DELIMITER $$

CREATE TRIGGER after_update
AFTER UPDATE
ON employee
FOR EACH ROW
BEGIN
    INSERT INTO employee_log(emp_name,action_name)
    VALUES(NEW.emp_name,'Employee Updated');
END $$
```

```
DELIMITER ;
```

```
UPDATE employee  
SET salary=65000  
WHERE emp_id=2;
```

```
SELECT * FROM employee_log;
```

```
-- Trigger 4  
DELIMITER $$
```

```
CREATE TRIGGER before_insert  
BEFORE INSERT  
ON employee  
FOR EACH ROW  
BEGIN  
    SET NEW.salary = IF(NEW.salary<20000,20000,NEW.salary);  
END $$
```

```
DELIMITER ;
```

```
INSERT INTO employee(emp_name,salary,dept_id,manager_id)  
VALUES('Riya',10000,2,1);
```

```
SELECT * FROM employee;
```

```
-- Trigger 5  
DELIMITER $$
```

```
CREATE TRIGGER before_update  
BEFORE UPDATE  
ON employee  
FOR EACH ROW  
BEGIN  
    SET NEW.salary = IF(NEW.salary<20000,20000,NEW.salary);  
END $$
```

```
DELIMITER ;
```

```
UPDATE employee  
SET salary=15000  
WHERE emp_id=1;
```

```
SELECT * FROM employee;
```

```
-- Trigger 6  
DELIMITER $$
```

```
CREATE TRIGGER before_delete
BEFORE DELETE
ON employee
FOR EACH ROW
BEGIN
    INSERT INTO employee_log(emp_name,action_name)
    VALUES(OLD.emp_name,'Before Delete');
END $$
```

```
DELIMITER ;
```

```
DELETE FROM employee
WHERE emp_id=4;
```

```
SELECT * FROM employee_log;
```

```
-- Show Triggers
SHOW TRIGGERS;
```

```
-- Show Employee Table
SELECT * FROM employee;
```

```
-- Show Log Table
SELECT * FROM employee_log;
```